

The updated USART initialization code is shown in Listing 8.14. We configure the MCU so the USART generates an interrupt after a character has been received or when the transmit buffer is empty (ready to accept a new character).

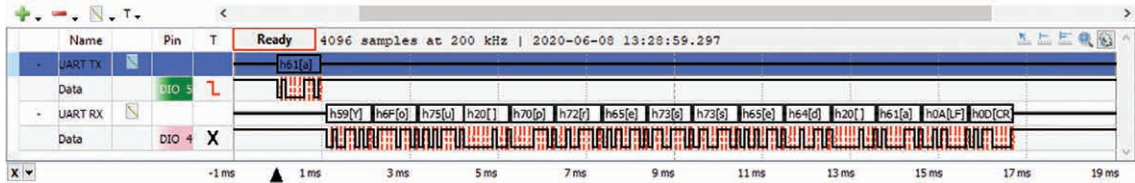


Figure 8.26 Logic analyzer displays received character “a” and resulting transmitted message “You pressed a”.

Figure 8.26 shows an example of the completed system’s communication activity. There is a slight delay (roughly 150 μ s) between the reception of the character and the transmission of the response.

Inter-Integrated Circuit Bus (I²C)

Protocol Concepts

I²C is a synchronous protocol that uses a serial data (SDA) signal and a serial clock (SCL) signal. I²C is a master/slave protocol. The master initiates all communications, and slave devices transmit only when the master allows them to. A major feature of the protocol is device addressing: each message includes device addressing information, and each device on the bus has a unique address. Only the addressed device will respond to a message. This allows a system to be built that shares the SDA/SCL bus as shown in Figure 8.27 without using additional control signals (such as slave selects for SPI).

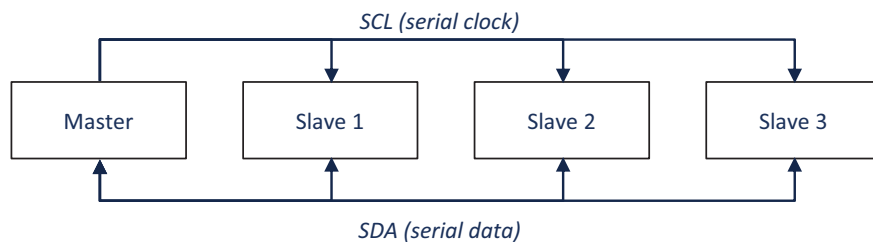


Figure 8.27 Overview of I²C system with master and three slave devices.

The circuits driving the SDA and SCL signals are shown in Figure 8.28 and are designed so that multiple devices can drive the signal simultaneously without damage. Each device’s drive circuit consists of a transistor connected between the I²C bus signal (SDA or SCL) and ground. A separate pull-up resistor is connected between the signal and V_{DD} . For the master to send a 0 on SDA,

the transistor Q2 is turned on, pulling SDA to ground. To send a one, the transistor Q2 is turned off, allowing the resistor R1 to pull SDA up to V_{DD} . If two devices attempt to transmit different data simultaneously, the SDA signal will be a zero.

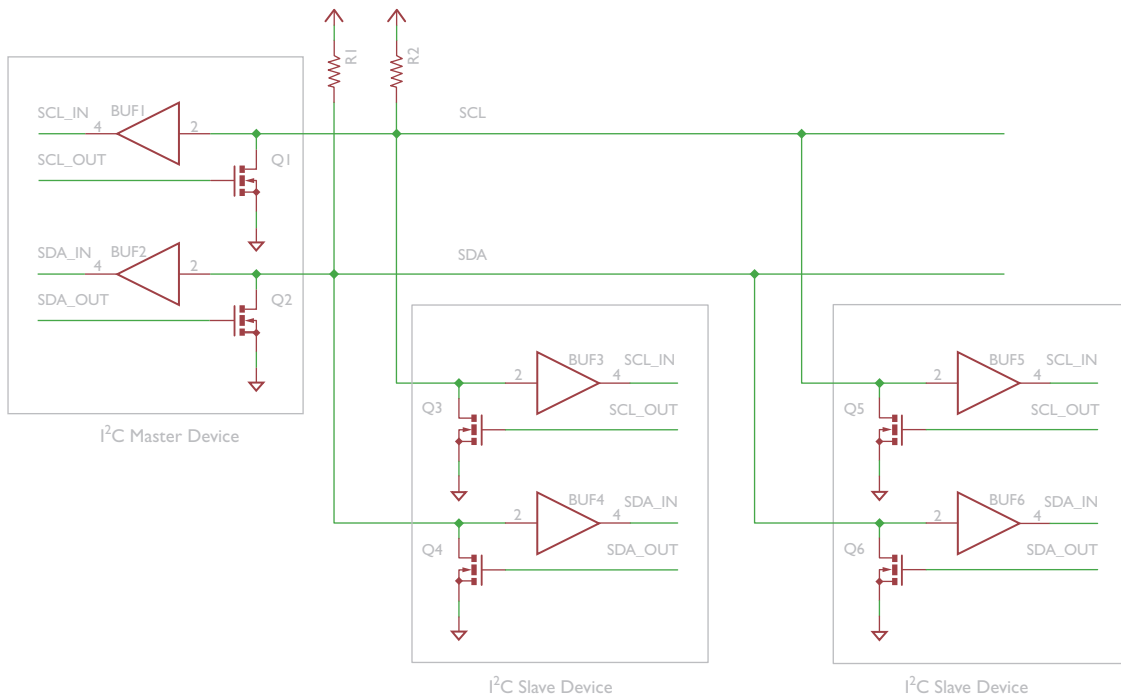


Figure 8.28 I²C signal drive circuits use “open drain” configuration to allow multiple devices to drive signals safely.

I²C supports a range of communication speeds: 100 kbit/s (standard), 400 kbit/s (full speed), 1 Mbit/s (fast), 3.2 Mbit/s (high speed). The maximum communication speed for a given system implementation is limited by how quickly the pull-up resistor can pull SCL or SDA up to V_{CC} . This depends on the capacitance of the SCL or SDA signal, which is affected by the number of devices and the bus length. The maximum speed for a given device will be listed in its data sheet or reference manual.

Message Format

There are several types of I²C message, with the basic format shown in Figure 8.29. Each message contains several fields:

- The start condition indicates the start of a message.
- The slave device address identifies the target of the communication.
- The read/write bit indicates whether the following data is to be read from the slave or written to it.

- The acknowledgment bit has two uses: to indicate if the addressed slave is present, and whether more data will be read. These are explained later in this section.
- One or more data bytes. For some I²C slave devices, the first data byte will be interpreted as a register address.
- A stop condition indicates the end of a message.
- An optional repeated start condition is used in some types of messages.

	Slave Device Address									Data Byte									
Start	AD7	AD6	AD5	AD4	AD3	AD2	AD1	R/W	ACK	D7	D6	D5	D4	D3	D2	D1	D0	ACK	Stop

Figure 8.29 Basic fields of a simple I²C message with 7-bit addressing and one data byte.

Communication operations are structured as sequences of conditions (e.g. start, stop) and data transfers (one byte and one acknowledgment bit). We will see this in the code example mentioned.

Device Addressing

The master uses the device address to select a particular slave device on the bus. There are two addressing modes, one with 7-bit addresses and another with 10-bit addresses. For simplicity we will just discuss the 7-bit mode.

The first byte in the message has two parts, as shown in Figure 8.29. The device address is held in the upper seven bits, and the R/W bit is the LSB. This bit indicates whether the master will read from the slave (one) or write to it (zero). In practice, this byte is formed by shifting the slave address left by one bit and then adding the R/W bit.

Master	Start	Slave Dev. Add.	W(0)		Data		Data		Stop
Slave				ACK(0)		ACK(0)		ACK(0)	

Figure 8.30 Message format for master writing two bytes to slave device. Text indicates transmitting device. Blank indicates listening device.

Figure 8.30 shows the operations involved for a master to write two data bytes to a slave device.

- The master first sends the start condition, the slave device address, and a write command (zero). If the addressed slave is present, it will assert the ACK bit. If the ACK bit is not asserted, then the master will terminate the message with a stop condition.
- The master sends the first byte of data.
- The slave sends an ACK to indicate it has been received.
- The master sends the second byte of data.
- The slave sends an ACK to indicate it has been received.
- The master sends the stop condition, indicating to all slaves that the message has completed.

Master	Start	Slave Dev. Add.	R(1)			ACK(0)		NACK(1)	Stop
Slave				ACK(0)	Data		Data		

Figure 8.31 Message format for master reading two bytes from slave device.

Figure 8.31 shows how the master can read two bytes from the slave.

- The master first sends the start condition, the slave device address, and a read command (one). If the addressed slave is present, it will assert the ACK bit. If the ACK bit is not asserted, then the master will terminate the message with a stop condition.
- The master clocks the first byte of data out of the slave.
- The master sends an ACK to indicate it will read more data.
- The master clocks the second byte of data out of the slave.
- The master sends a NACK (one) to indicate that it does not want to read any more data in this message.
- The master sends the stop condition, indicating to all slaves that the message has completed.

Register Addressing

I²C also supports register addressing, in which each device is structured as a series of addressable registers that can be read or written. This standardizes information organization and simplifies system development. The first data byte is interpreted by the slave device as a register address. Figures 8.32 and 8.33 show examples of writing to and reading from a register in a device.

Master	Start	Slave Dev. Add.	W(0)		Slave Reg. Add.		Data		Stop
Slave				ACK(0)		ACK(0)		ACK(0)	

Figure 8.32 Message format for master writing one byte to a specific register in the slave device.

Master	Start	Slave Dev. Add.	W(0)		Slave Reg. Add.		Start	Slave Dev. Add.	R(0)			NACK(1)	Stop
Slave				ACK(0)		ACK(0)				ACK(0)	Data		

Figure 8.33 Message format for master reading one byte from a specific register in the slave device.

I²C devices with register addressing may offer an **auto-increment** mode for the register address. With this mode, a write message with multiple data bytes will write the first data byte to the specified register address, the second byte to the next address, and so forth. Reads are similar. This mode reduces timing overhead and software complexity.

STM32F091RC I²C Peripherals

The STM32F091RC microcontroller has two I²C peripherals, called I2C1 and I2C2. Each I2Cx peripheral contains a baud rate generator, a shift-register-based transmitter/receiver, bus interface circuitry, and extensive control logic. I2C1 offers more features than I2C2 (see Section 26.3 of the reference manual), but we will not use those features here.

The hardware handles each message as a sequence of bytes. The software must also be structured accordingly, as we will see shortly.

General Control

Clock gating must be enabled for the appropriate I2Cx peripheral using the RCC register RCC_APB1EN.

The I²C control register one (I2Cx_CR1, shown in Figure 8.34) controls various aspects of the peripheral's operation. The relevant fields for master mode are these:

- PE enables the peripheral to operate.
- ERRIE, TCIE, STOPIE, NACKIE, ADDRIE, RXIE, and TXIE enable interrupts for the peripheral.
- TXDMAEN and RXDMAEN (bits 14 and 15 of I2Cx_CR1) enable DMA transfers.
- The I²C control register two (I2Cx_CR2) has the following relevant field in master mode:
- SADD contains the slave address and ADD10 indicates if the address is on 7 bits (zero) or on 10 bits (one).
- RD_WRN selects if the peripheral will transmit (zero) or receive (one).
- NACK controls whether to transmit an ACK (zero) or a NACK (one) after a byte is received.
- NBYTES indicates the number of bytes to transfer.

Bit	7	6	5	4	3	2	1	0
Field	ERRIE	TCIE	STOPIE	NACKIE	ADDRIE	RXIE	TXIE	PE
Reset	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw

Figure 8.34 Contents of I2Cx_CR1 control register (only bits 7 to 0 are shown on the figure).

Status and Interrupts

The I²C Interrupt and Status Register (I2Cx_ISR, shown in Figure 8.35) indicates the status of the peripheral. The relevant fields for master mode are these:

- TC indicates that a byte and acknowledgment bit transfer has completed.
- BUSY indicates the bus is busy.
- RXNE indicates that the receive buffer is not empty and TXE indicates that the transmit buffer is empty.
- NACKF indicates that an acknowledgment bit was received (one) after transmitting a byte. A zero indicates no acknowledgment was received.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	BUSY	Res.	ALERT	TIME OUT	PECERR	OVR	ARLO	BERR	TCR	TC	STOPF	NACKF	ADDR	RXNE	TXIS	TXE
Reset	0		0	0	0	0	0	0	0	0	0	0	0	0	0	1
Access	r		r	r	r	r	r	r	r	r	r	r	r	r	r*	r*

Figure 8.35 Contents of I2Cx_ISR interrupt and status register (only the bits 15 to 0 are shown on the figure).

The I²C Transmit Data Register (I2Cx_TDR) holds data to transmit and the Receive Data Register (I2Cx_RDR) holds the received data.

Baud Rate Generator

The communication speed (I²C baud rate) can be chosen between three modes: standard mode (up to 100 KHz), fast mode (up to 400 KHz) or fast mode plus (up to 1 MHz). The I2C1 peripheral can use the bus clock, the system clock or the HSI oscillator as its input clock.

The Timing Register (I2Cx_TIMINGR) needs to be configured, however the computation of its value is not trivial. STMicroelectronics provides a tool (presented in [7]) to help compute its value. The HAL API may simplify this process by automatically programming the I2Cx_TIMINGR register to the correct value depending on the parameters chosen for some development boards.

Other Features

The I²C peripheral has many other features not covered here: the ability to operate as a slave (with an address match comparison), DMA, general call messages, and system management bus (SMB) support. There is a low-power mode that can operate while the rest of the MCU is in sleep mode.

Example: Polled I²C Communications with an Inertial Sensor

Let's use I²C to configure and monitor an inertial sensor. If the sensor is within 2° of horizontal, the green LED is lit. If not, the other LEDs are lit indicating too much roll (red) or pitch (blue). We use polling code with busy-waiting to simplify the implementation and explanation. An interrupt-driven approach would be more efficient but more complex, and therefore a good assignment for students.

The X-NUCLEO-IKS01A1 is an expansion board that can be plugged into the Nucleo-F091RC board. It includes a three-axis accelerometer and a three-axis gyroscope (LSM6DSL [8]) that detects acceleration and gravity, a humidity and temperature sensor (HTS221), a three-axis magnetometer (LIS3MDL) that detects magnetic fields and a pressure sensor (LPS25HB). We will use the LSM6DSL 3-axis accelerometer to determine the inclination of the board in space. The sensor is connected to the MCU's I2C1 I²C bus, on pins PB9 (SDA) and PB8 (SCL), as described in the documentation (Section 3.6 Connectors in [9] and Section 6.11 Arduino connectors in [10]). The SCL and SDA signals are pulled up to 3.3 V with pull-up resistors. The sensor also has one output on PB5 that can trigger MCU interrupt requests, to indicate that a certain condition has occurred (shown in Figure 8.36).

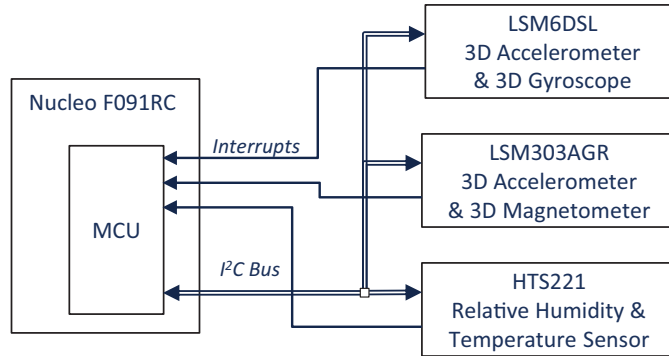


Figure 8.36 Connections between Nucleo-F091RC and X-NUCLEO-IKS01A1.

Let's see how to access the accelerometer through I²C using the peripheral I2C1. There is also a driver interface for LSM6DSL provided by STMicroelectronics available, but we do not use it because it hides the details of the communication [6].⁶

Initialize I2C1 Peripheral

The `I2C_Init` function shown in Listing 8.13 initializes the GPIO port B pins and the I2C1 peripheral. The peripheral clocks for I2C1 and GPIOB are enabled, and then PB8 and PB9 are configured for the I2C1 alternate function. Default values for the control registers CR1 and CR2 are loaded, as well as a value for TIMINGR derived from the example code A.14.1 in the reference manual [5]. Finally, the peripheral is enabled.

```
void I2C_Init(void) {
    // Clock gating for I2C1 and GPIO B
    RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;
    RCC->AHBENR  |= RCC_AHBENR_GPIOBEN;

    // GPIO B pin 8 and 9 in alternate function
    MODIFY_FIELD(GPIOB->MODER, GPIO_MODER_MODER8, ESF_GPIO_MODER_ALT_FUNC);
    MODIFY_FIELD(GPIOB->MODER, GPIO_MODER_MODER9, ESF_GPIO_MODER_ALT_FUNC);
    // Select I2C1 (AF = 1) as alternate function
    MODIFY_FIELD(GPIOB->AFR[1], GPIO_AFRH_AFSEL8, 1);
    MODIFY_FIELD(GPIOB->AFR[1], GPIO_AFRH_AFSEL9, 1);

    // I2C1 Configuration
    I2C1->CR1 = 0; // Default configuration, peripheral disabled
    I2C1->CR2 = 0; // Default configuration, 7 bit addressing
    I2C1->TIMINGR = 0x00B01A4B; // about 71 kbaud

    MODIFY_FIELD(I2C1->CR1, I2C_CR1_PE, 1); // Enable peripheral
}
```

Listing 8.15 Code to initialize I2C1 peripheral and pins PB8 and PB9.

⁶ To use the interface for the driver, the files `lsm6dso_reg.h`, `lsm6dso_reg.c` need to be downloaded from STMems_Standard_Driver Github repository and added the project. Several examples of I²C are available in the Appendix of the Reference Manual [5] and several examples of the LSM6DSL sensor are available in the Github repository.

The code first the I2C1 peripheral to send a START condition followed by the device address with the write flag set. The code then sends the register address and awaits its completion. It then loops to send each data byte, blocking after each byte until the transmitter is empty. Finally, the code sends the STOP condition to end the I²C transaction.

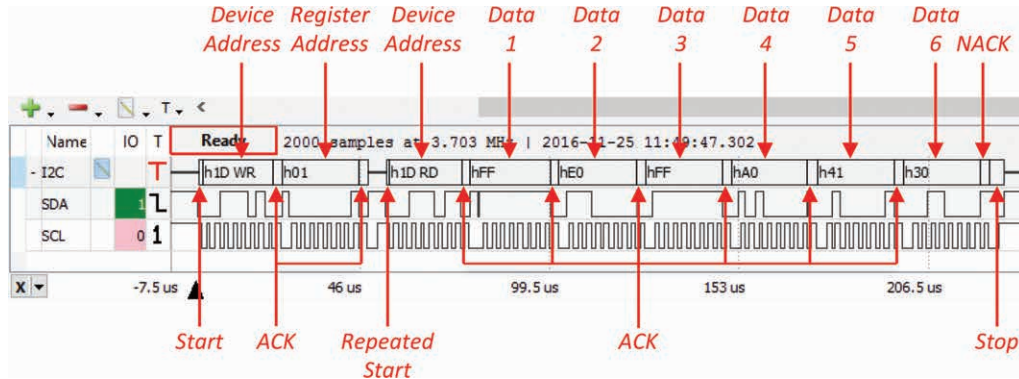


Figure 8.38 Example of signals and sequence of operations to read multiple bytes.

Read Data

Reading is a little more complex than writing; it consists of writing the device and register addresses and then reading the register data from the slave. Listing 8.17 shows the code to read multiple data bytes from a specific device register. Figure 8.38 shows an example of bus activity from the logic analyzer to read data from six consecutive slave registers.

```
void I2C_ReadReg(uint8_t dev_adx, uint8_t reg_adx, uint8_t *bufp,
    uint16_t data_len) {
    uint32_t tmp;
    // -- Send START, Device Address, Write Command --
    tmp = 0;
    MODIFY_FIELD(tmp, I2C_CR2_SADD, dev_adx << 1);
    MODIFY_FIELD(tmp, I2C_CR2_RD_WRN, 0); // First write addresses
    MODIFY_FIELD(tmp, I2C_CR2_NBYTES, 1); // 1 byte: register address
    MODIFY_FIELD(tmp, I2C_CR2_START, 1); // Start transfer
    I2C1->CR2 = tmp;
    while (I2C1->CR2 & I2C_CR2_START) // Wait until START ends
        ;
    // -- Write Register Address --
    I2C1->TXDR = reg_adx;
    while (!(I2C1->ISR & I2C_ISR_TXE)) // Wait until sent
        ;
    // -- Send Repeated START, Device Address, Read Command--
    tmp = I2C1->CR2;
    MODIFY_FIELD(tmp, I2C_CR2_SADD, dev_adx << 1);
    MODIFY_FIELD(tmp, I2C_CR2_RD_WRN, 1); // Then read data
    MODIFY_FIELD(tmp, I2C_CR2_NBYTES, data_len); // Data byte count
    MODIFY_FIELD(tmp, I2C_CR2_START, 1); // Start transfer
    I2C1->CR2 = tmp;
```

```

// -- Read Data --
while (data_len--) {
    while (!(I2C1->ISR & I2C_ISR_RXNE)) // Await data rx
        ;
    *bufp++ = I2C1->RXDR; // Read RXDR, advance pointer
}
// -- Write Stop --
MODIFY_FIELD(I2C1->CR2, I2C_CR2_STOP, 1);
}

```

Listing 8.17 Source code to read data from I²C device register.

The code begins by sending the START condition followed by the device address with the write command. The code then sends another START (called a repeated start) followed by the device address with a read command. The code then reads each requested byte of data from the slave. Finally, the code sends the STOP condition to end the transaction.

Initialize LSM6DSL

After the I2C1 peripheral has been initialized, we need to initialize the LSM6DSL inertial module. We do this by using I²C to access the module's registers, some of which are shown in Table 8.1.

Table 8.1 Selected LSM6DSL registers, from sensor datasheet [8].

Name	Type	Register Address	Comment
WHO_AM_I	Read-only	0x0F	Who am I ID, returns 0x6A when read
CTRL1_XL	Read	0x10	Accelerometer sensor control register 1
CTRL3_C	Read/Write	0x12	Control register 3
OUTX_L_XL	Read/Write	0x28	X accelerometer output, least-significant byte
OUTX_H_XL	Read/Write	0x29	X accelerometer output, most-significant byte
OUTY_L_XL	Read/Write	0x2A	Y accelerometer output, least-significant byte
OUTY_H_XL	Read/Write	0x2B	Y accelerometer output, most-significant byte
OUTZ_L_XL	Read/Write	0x2C	Z accelerometer output, least-significant byte
OUTZ_H_XL	Read/Write	0x2D	Z accelerometer output, most-significant byte

First, we check to see if the module is present on the I²C bus. According to the module's datasheet, reading its WHO_AM_I register should return a code of 0x6A. If that code is not returned, communication with the module has failed so the program halts.

The module's datasheet describes the default settings after the device is powered up. Most of the default settings meet our needs, but we will change some using the initialization code in Listing 8.18. The datasheet explains how to determine which values to write to which module registers.

We select a data rate of 104 Hz and a full-scale sensitivity from $-2 g_n$ to $+2 g_n$. Note that g_n is the acceleration ($32 f/s^2$ or $9.8 m/s^2$) due to standard gravity. With this sensitivity, the LSB of each acceleration measurement represents $0.000061 g_n$. We also select register address auto-increment and block data update (to prevent data corruption when reading).

```

#define LSM6DSL_DEV_ADX          (0x6b)
#define LSM6DSL_REG_ADX_WHO_AM_I (0x0f)
#define LSM6DSL_WHO_AM_I_CODE   (0x6a)

#define LSM6DSL_REG_ADX_CTRL1_XL (0x10)
#define LSM6DSL_REG_ADX_CTRL3_C  (0x12)
#define LSM6DSL_REG_ADX_OUTX_L_XL (0x28)

void Init_Inertial_Sensor(void) {
    uint8_t data;

    I2C_ReadReg(LSM6DSL_DEV_ADX, LSM6DSL_REG_ADX_WHO_AM_I, &data, 1);
    if (data != LSM6DSL_WHO_AM_I_CODE)
        while (1); // Halt: sensor not found

    // CTRL3
    // IF_INC = 1: Enable auto-increment register address
    // BDU = 1: Enable block data update
    data = 0x44;
    I2C_WriteReg(LSM6DSL_DEV_ADX, LSM6DSL_REG_ADX_CTRL3_C, &data, 1);

    // CTRL1_XL
    // FS_XL = 00: 2G full scale
    // ODR_XL = 0100: 104Hz data rate
    data = 0x40;
    I2C_WriteReg(LSM6DSL_DEV_ADX, LSM6DSL_REG_ADX_CTRL1_XL, &data, 1);
}

```

Listing 8.18 Initialization of LSM6DSL to use the accelerometer with I²C communication.

Get and Use Acceleration Data

The function `read_full_xyz` in Listing 8.19 gets the acceleration data from the inertial module. As shown in Table 8.1, there are six consecutive one-byte registers with the acceleration data for the X, Y and Z axes in LSB and MSB order. We could get the acceleration data by performing six separate I²C single byte read operations, but that would be slow. We selected the module's auto-increment register address feature during initialization, so instead we can perform one six-byte read operation (starting with the first register, `LSM6DSL_REG_ADX_OUTX_L_XL`), saving time.

The X, Y and Z accelerations are then calculated by scaling the raw values by the accelerometer's sensitivity per bit. The accelerations are 0.001 g_n per bit, so a reading of 1000 represents one standard gravity g_n . The `g_z` variable should be about 1000, assuming the board is horizontal, face up and not accelerating vertically. For greater accuracy, we would compensate for offset and other errors.

```

#define SENSITIVITY_AT_2G (0.061f) // mg per LSB
#define M_PI              (3.14159f)

float g_roll, g_pitch;
int32_t g_x, g_y, g_z;

void read_full_xyz(void) {
    int16_t raw_accel[3];

    I2C_ReadReg(LSM6DSL_DEV_ADX, LSM6DSL_REG_ADX_OUTX_L_XL,
        (uint8_t *) raw_accel, 6);
}

```

```

    /* Convert the data. */
    g_x = (int32_t)((float)raw_accel[0] * SENSITIVITY_AT_2G);
    g_y = (int32_t)((float)raw_accel[1] * SENSITIVITY_AT_2G);
    g_z = (int32_t)((float)raw_accel[2] * SENSITIVITY_AT_2G);
}

void convert_xyz_to_roll_pitch(void) {
    g_roll = atan2f(g_y, g_z)*180/M_PI;
    g_pitch = atan2f(-g_x, sqrtf(g_y*g_y + g_z*g_z))*180/M_PI;
}

```

Listing 8.19 Function `read_full_xyz` reads 6 bytes of data from sensor. Function `convert_xyz_to_roll_pitch` calculates roll and pitch based on accelerations.

The function `convert_xyz_to_roll_pitch` (also in Listing 8.19) uses trigonometry to calculate the global variables `g_roll` and `g_pitch`. Single-precision floating-point math functions (`atan2f` and `sqrtf`) are used rather than the slower but more accurate double-precision versions (`atan2` and `sqrt`).

```

#define ROLL_MAX_DEG (2.0)
#define PITCH_MAX_DEG (2.0)

int main(void) {
    int red, green, blue;

    Set_Clocks_To_48MHz();
    Init_GPIO_RGB();
    Control_RGB_LEDs(0,0,0);
    I2C_Init();
    Init_Inertial_Sensor();
    while (1) {
        read_full_xyz();
        convert_xyz_to_roll_pitch();
        red = (g_roll > ROLL_MAX_DEG) || (g_roll < -ROLL_MAX_DEG)? 1 : 0;
        blue = (g_pitch > PITCH_MAX_DEG) || (g_pitch < -PITCH_MAX_DEG)? 1 : 0;
        green = (red | blue)? 0 : 1;
        Control_RGB_LEDs(red, green, blue);
    }
}

```

Listing 8.20 `main` function sets up peripherals and sensor, then lights LEDs based on sensor's roll and pitch.

The `main` function shown in Listing 8.20 initializes the peripherals and the inertial module. It then repeatedly reads the X, Y, Z accelerations, calculates roll and pitch, and then lights the LEDs based upon the board's inclination.

Summary

In this chapter, we have seen the motivation for communicating information serially and the core issues that must be tackled to do so. We examined helpful development tools and software structures. We have studied three different types of communication protocols (synchronous serial, asynchronous serial, and I²C) and how to implement them using the peripherals of the MCU.

Exercises

1. Examine Chapter 4 (Pinouts and pin descriptions) of the STM32F091RC data sheet to determine the answers to the following questions. Assume that an MCU in an LQFP64 package is used.
 - a. Which port bits can be used for SPI1?
 - b. Which port bits can be used for SPI2?
2. Show the register settings needed to configure SPI1 to operate as a master at 12 MHz, 8 data bits (MSB first), SPI mode zero (Clock Phase CPHA = 0, Clock Polarity CPOL = 0). Assume the bus clock is 24 MHz. Enable interrupts for transmission, reception, and errors. Use the NSS pin as a slave select output.
3. Draw a timing diagram showing the bytes 0x31 0xF1 being transmitted by SPI at 1,000,000 baud, with SPI mode zero. Indicate the time of each signal transition.
4. Examine Chapter 4 (Pinouts and pin descriptions) of the STM32F091RC data sheet to determine the answers to the following questions. Assume that an MCU in an LQFP64 package is used.
 - a. Which port bits can be used for USART1?
 - b. Which port bits can be used for USART2?
 - c. Which port bits can be used for USART3?
5. Show the register settings needed to configure USART1 to transmit and receive at 71,433 baud, eight data bits (LSB first), one stop bit and odd parity. Assume the bus clock is 24 MHz. Enable interrupts to indicate that the transmit data register is empty, the receive data register is full, or any error has occurred. The USART should not trigger any DMA activity.
6. Assume a USART has both TXEIE and TCIE set to one and a program writes a byte to the USARTx_TDR register for transmission. Which interrupts will occur, and when?
7. Draw a timing diagram showing the bytes 0x31 0xF1 being transmitted by a USART at 115,200 baud, with LSB first, odd parity, and one stop bit. Indicate the time of each signal transition.
8. Examine Chapter 4 (Pinouts and pin descriptions) of the STM32F091RC data sheet to determine the answers to the following questions. Assume that an MCU in an LQFP64 package is used.
 - a. Which port bits can be used for I2C1?
 - b. Which port bits can be used for I2C2?
9. Draw a timing diagram of the following I²C message: a value of 0x31 being written to device 0x36 register 0x55. Assume 200 kbaud communications speed. Indicate the time of each signal transition.

References

- [1] International Telecommunication Union, “Data Networks and Open System Communications: Open Systems Interconnection – Model and Notation,” 1994.
- [2] P. J. Koopman and T. C. Maxino, “The Effectiveness of Checksums for Embedded Control Networks,” IEEE Transactions on Dependable and Secure Computing , vol. 6, no. 1, pp. 59 -72, 2009.
- [3] Digilent, Inc., “Analog Discovery 2 [Reference.DigilentInc],” [Online]. Available: <https://reference.digilentinc.com/reference/instrumentation/analog-discovery-2/start>. [Accessed June 11, 2020].
- [4] Dangerous Prototypes, Where Labs, LLC., “Bus Pirate --DP,” [Online]. Available: http://dangerousprototypes.com/docs/Bus_Pirate. [Accessed June 11, 2020].
- [5] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.
- [6] STMicroelectronics NV, *User Manual UM1785: Description of STM32F0 HAL and Low-Layer Drivers*, 2020. [Online]. Available: https://www.st.com/resource/en/user_manual/dm00122015-description-of-stm32f0-hal-and-lowlayer-drivers-stmicroelectronics.pdf.
- [7] STMicroelectronics NV, *Application note AN4235: I²C Timing Configuration Tool for STM32F3xxxx and STM32F0xxxx Microcontrollers*, DocID 024161, rev. 2, 2013.
- [8] STMicroelectronics NV, *LSM6DSL Datasheet: iNEMO Inertial Module: Always-On 3D Accelerometer and 3D Gyroscope*, DocID 028475, 2017.
- [9] STMicroelectronics NV, *User Manual UM2121: Getting Started with the X-NUCLEO-IKS01A2 Motion MEMS and Environmental Sensor Expansion Board for STM32 Nucleo*, DocID 029834, rev. 2, 2017.
- [10] STMicroelectronics NV, *User Manual UM1724: STM32 Nucleo-64 Boards*, 2019.

